Event Contract Guard

A working prototype: onboard a new processor or ledger with a declarative mapping, validate every event against a canonical contract, and catch schema drift and missing events before they hit reconciliation.

The problem

Every processor, bank and ledger sends events in its own shape. Two failure modes are quiet and expensive: **(1) schema drift**, where a provider renames a field or changes a type without notice, and **(2) missing events**, where a payment has a later lifecycle stage but an earlier one never arrived. Both usually surface late, as reconciliation breaks that an operator has to trace by hand.

How it works

Layer	What it does
1. Canonical contract	One schema per event type (authorized, captured, settled) with required fields and types.
2. Adapter (config only)	A new processor is onboarded by declaring its event-type map and a canonical-field to payload-path map (dotted paths for nested JSON). No new code.
3. Validation	Every event is normalized to the canonical shape. Missing fields and wrong types are flagged with the exact field and value.
4. Drift detector	Learns a per-processor, per-event baseline of (field path, type) from the first N events, then flags new fields (info), removed fields (warn) and type changes (critical).
5. Lifecycle checker	Per payment, verifies authorized then captured then settled. Flags gaps (a later stage exists but an earlier one is missing) and stuck payments.

Onboarding a processor is just config

```
Adapter("proc_b", "event",
    {"AUTHORIZED": "payment.authorized", "CAPTURED": "payment.captured",
     "SETTLED": "payment.settled"},
    {"payment_id": "data.txn_id", "amount_minor": "data.amount_minor",
     "currency": "data.currency", "timestamp": "created_at",
     "settlement_batch": "data.batch_id"})
```

Demo run (simulated data)

The demo streams 60 payments (20 each from three simulated processors, 3 lifecycle events per payment) with faults injected on purpose. The output below is the real output of `demo.py` (long lines wrapped for the page).

- **proc_a**: from payment 12 onward the amount silently changes from int to string, and `ccy` is renamed to `currency`.
- **proc_b**: a new field `data.risk_score` appears. Harmless, so it is reported as info only.
- **proc_c**: no `captured` event for payment 4, and no `settled` event for payments 9 and 15.

```
=== EVENT CONTRACT GUARD - RUN REPORT ===
canonical events accepted : 153
issues raised             : 159

--- summary by processor / issue kind ---
proc_a  drift_missing_field    x24
proc_a  drift_new_field        x24
proc_a  drift_type_change      x24
proc_a  validation_missing     x24
proc_a  validation_type        x24
proc_b  drift_new_field        x36
proc_c  missing_event          x1
proc_c  stuck_payment          x2

--- first occurrence of each issue kind ---
[CRITICAL] proc_a drift_type_change: payment.authorized: 'amount' changed int -> str
[CRITICAL] proc_a validation_type: payment.authorized: 'amount_minor' expected int, got str ('10.12')
[CRITICAL] proc_a validation_missing: payment.authorized: 'currency' missing (mapped from 'ccy')
[CRITICAL] proc_c missing_event: payment PC004: 'payment.captured' never received but later stage
           'payment.settled' exists
[WARN    ] proc_a drift_missing_field: payment.authorized: field 'ccy' no longer sent
[WARN    ] proc_c stuck_payment: payment PC009: stopped at 'payment.captured', no 'payment.settled' yet
[INFO    ] proc_a drift_new_field: payment.authorized: new field 'currency' (str)
[INFO    ] proc_b drift_new_field: payment.authorized: new field 'data.risk_score' (float)
```

What this shows

- The type change is caught as **critical** at the first bad event, not after a reconciliation break.
- The rename is reported three ways (old field gone, new field seen, canonical field missing), so the cause is obvious.
- The gap in proc_c is pinpointed to the exact payment and the exact missing stage. The two stuck payments are flagged as warnings, not errors, since settlement may simply not have happened yet.

Core drift logic (excerpt from guard.py)

```
# learn baseline from the first N events per (processor, event type)
if self.seen_count[key] <= self.baseline_size:
    for p, t in shape.items():
        self.baseline[key].setdefault(p, t)
else:
    base = self.baseline[key]
    for p in sorted(set(shape) - set(base)):          # new field   -> info
        self._issue("drift_new_field", "info", ...)
    for p in sorted(set(base) - set(shape)):          # field gone  -> warn
        self._issue("drift_missing_field", "warn", ...)
    for p in sorted(set(base) & set(shape)):          # type change -> critical
        if base[p] != shape[p]:
            self._issue("drift_type_change", "critical", ...)
```

Honest limitations

- Runs on **simulated** events. It has not been tested on real processor traffic.
- Baseline learning assumes the first N events are healthy. A production version needs a versioned, human-approved baseline.
- Issues are not yet de-duplicated or grouped, so one root cause can create many alerts (see counts above).
- Lifecycle checking has no time windows or SLAs yet, so it cannot tell 'late' from 'lost'.

Where I would take it next

- Wrap the engine in a FastAPI service: `POST /events` to ingest, `GET /issues` to read, plus a dry-run endpoint that validates a sample payload while onboarding a new processor.
- Group alerts into one incident per root cause, with a sample payload as evidence.
- Use an LLM to draft the adapter mapping from a processor's API docs or sample payloads, with a human confirming it, and to explain drift in plain language.
- Add SLA windows per lifecycle stage and persistent storage for baselines and issues.

Code: [GitHub link] | [Your name], final-year engineering student | [email] | [LinkedIn]